Süreçler (Processes) Yönetimi ve Temel Kavramlar

1. Proses Nedir? (Program vs. Proses)

En temelden başlayalım:

- **Program:** Bilgisayarında duran (hard diskte) pasif bir kod yığınıdır (Örn: `chrome.exe` dosyası). Ne zaman ki sen ona çift tıklarsın ve o belleğe (RAM) yüklenip çalışmaya başlar, işte o zaman adı Proses olur.
- **Proses:** Programın belleğe (RAM) yüklenip çalışmaya başlamış halidir.

Kavram	Özellik
Program	Pasiftir, depolama alanında bekler.
Proses	Aktiftir, CPU ve bellek kullanır.

Process Kavramı

Bir işletim sistemi çeşitli programları yürütür. İlk bilgisayarlar, işleri toplu yürüten batch sistemlerdi, ardından kullanıcı programlarını veya görevlerini time-shared (zaman paylaşımlı) olarak çalıştıran sistemler ortaya çıktı.

- **Batch systems:** Batch sisteminin kullanıcıları, bilgisayarla doğrudan etkileşime girmezlerdi. Her kullanıcı işini delikli kartlar (punch cards) gibi çevrimdışı bir cihazda hazırlar ve bilgisayar operatörüne sunardı. İşlemeyi hızlandırmak için benzer ihtiyaçlara sahip işler bir araya toplanır ve grup olarak çalıştırılırdı. Batch sistemlerde mekanik I/O cihazlarının hızı CPU'dan çok yavaş olduğu için CPU genellikle boştadır.
- **Time-shared:** Birden fazla proses, CPU tarafından aralarında geçiş yapılarak yürütülür, ancak bu geçişler o kadar sık gerçekleştirilir ki kullanıcı bunu fark etmez. Böylece CPU'nun boşta kaldığı zaman azaltılmış olur.

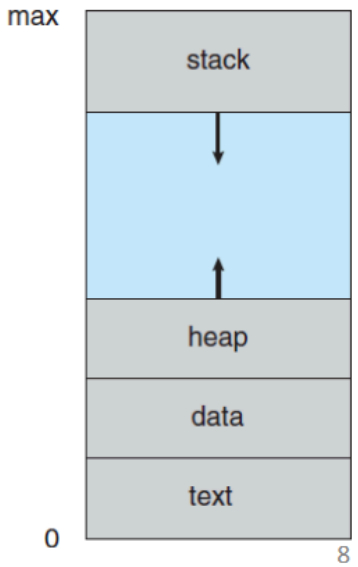
Günümüz işletim sistemleri, birden çok programın hafızaya yüklenmesine ve eşzamanlı çalıştırılmasına izin verir.

Çalışmakta olan programa proses denilmektedir.

CPU, aralarında geçiş yaparak tüm prosesleri eş zamanlı çalıştırılabilir.

Bir sistem, tek kullanıcılı bile olsa, birden fazla uygulamayı (Word, Excel, Web Browser, ...) birlikte çalıştırabilir.

- Bir prosesin anlık durum bilgisi program counter ve process registers ile tutulur.
  - program counter - instruction pointer olarak da bilinir, prosesin bir sonraki yürüteceği komutunun adresini tutar.
  - process registers - bir interrupt olduğunda prosesin register bilgileri kaydedilir, proses tekrar running durumuna geçtiğinde register bilgileri yeniden yüklenerek en son kaldığı yerden devam eder.
- Bir prosesin bileşenleri:
  - text section - program kodudur
  - data section - global değişkenleri saklar.
  - heap - prosese runtime'da dinamik olarak ayrılan bellektir.
  - stack - fonksiyonlar çağrıldığında geçici verileri saklar: fonksiyon parametreleri, return adresleri, lokal değişkenler vb.



stack ve heap sections, program yürütülürken dinamik olarak büyüyüp küçülebilir.

- Bir fonksiyon her çağrıldığında, fonksiyon parametrelerini, yerel değişkenleri ve dönüş adresini içeren bir activation record, stack'a eklenir; fonksiyondan geri dönüldüğünde, stack'tan activation record çıkarılır.
- Benzer şekilde, bellek dinamik olarak tahsis edildikçe heap büyüyecek ve bellek sisteme geri döndüğünde küçülecektir.

stack ve heap sections birbirine doğru büyümesine rağmen, işletim sistemi bunların birbiriyle çakışmamasını sağlamalıdır.

> 💡 ***Mantıksal Benzetme:*** *Programı bir yemek tarifi kitabı gibi düşün. Kitap rafta dururken kimseyi doyurmaz (pasif). Proses ise o tarife bakarak mutfakta yemek pişirme eylemidir. Ocak (CPU) kullanılır, malzemeler (Hafıza) harcanır ve ortaya bir ürün çıkar (aktif).*

Proses ile Program Arasındaki Farklar

Program	Proses
Program, belirli bir görevi tamamlamak için tasarlanmış bir dizi komut içerir.	Proses, yürütülmekte olan bir programın bir örneğidir.
Program, ikincil bellekte bulunduğu için **pasif** bir varlıktır.	Proses, yürütülme sırasında oluşturulduğu için ve ana belleğe yüklendiği için **aktif** bir varlıktır.
Programın herhangi bir kaynak gereksinimi yoktur, yalnızca komutların saklanması için depolama alanı gerektirir.	Prosesin yüksek kaynak gereksinimi vardır, ömrü boyunca CPU, bellek adresi, I/O gibi kaynaklara ihtiyaç duyar.
Programın herhangi bir kontrol bloğu yoktur.	Proses, Process Control Block (PCB) adı verilen kendi kontrol bloğuna sahiptir.

---

2. Bir Prosesin Bellek Yapısı (Memory Layout)

Bir programı çalıştırdığında, işletim sistemi ona belli bir bellek alanı ayırır. Ancak bu alanın içindeki **Stack** ve **Heap** kısımları sabit değildir; programın o anki ihtiyacına göre değişirler.

Bölüm	Özellikler
Text	Programın kodu burada durur (Sabittir).
Data	Global ve statik değişkenler buradadır (Sabittir).
Heap (Öbek)	Çalışma anında (runtime) dinamik olarak ayrılan bellek (Örn: `malloc()` ile alınan yer). İsteğe göre büyür veya küçülür. Dinamik bellek ayırma burada yapılır.
Stack (Yığın)	Fonksiyon çağrıları, yerel değişkenler ve dönüş adresleri/parametreleri burada tutulur. "Geçici iş listesi" gibidir. Proses çalıştıkça büyür ve küçülür.

"Birbirine Doğru Büyümek" Ne Demek?

Bu kısımları birbirine zıt yönlerden büyüyen iki grup gibi hayal et. Genelde Stack yukarıdan aşağıya, Heap ise aşağıdan yukarıya doğru genişler. Ortadaki boş alan hangisine lazımsa o oraya doğru genişler.

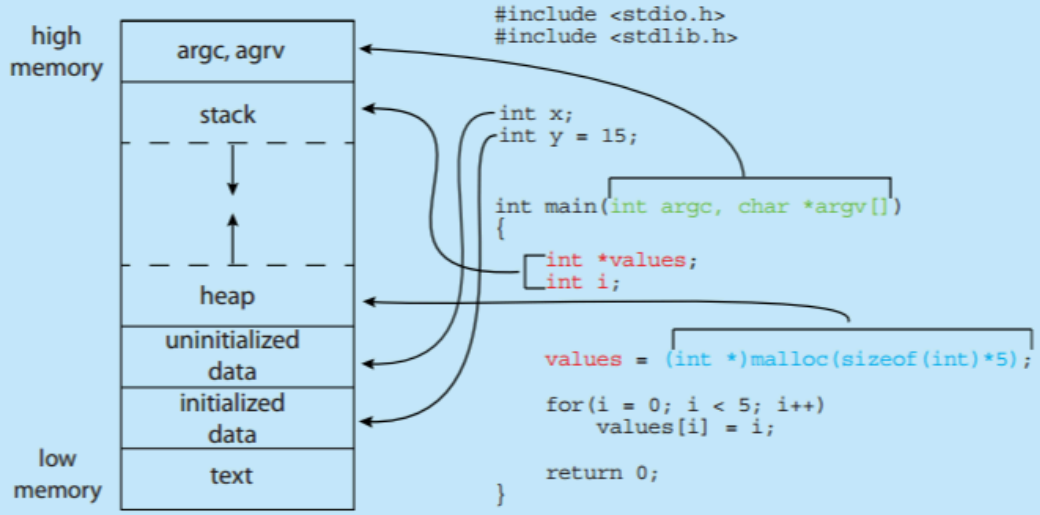
- **Neden böyle?** Belleği en verimli bu şekilde kullanırız.
- **İşletim Sisteminin Görevi:** Bu ikisi ortada buluşup birbirinin üstüne binmemelidir. Eğer çakışırlarsa sistem çöker (meşhur **Stack Overflow** hatası).

> 🧠 ***Akılda Kalıcı Kodlama: "KAB" Formülü*** *Prosesin ihtiyaç duyduğu 3 temel şey:*
>
> *1. **K**aynak (CPU, I/O)*
> *2. **A**dres Alanı (Bellek)*
> *3. **B**lok (PCB - Process Control Block)*

MEMORY LAYOUT OF A C PROGRAM

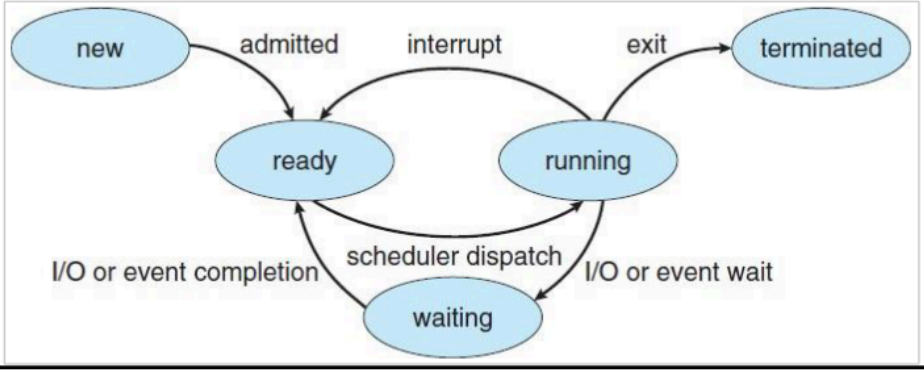
The figure shown below illustrates the layout of a C program in memory, highlighting how the different sections of a process relate to an actual C program. This figure is similar to the general concept of a process in memory as shown in Figure 3.1, with a few differences:

- The global data section is divided into different sections for (a) initialized data and (b) uninitialized data.
- A separate section is provided for the `argc` and `argv` parameters passed to the `main()` function.



Bir proses çalıştığı sürece durum değiştirir.

- New: Proses oluşturulmaktadır.
- Running: Komutlar çalıştırılmaktadır.
- Waiting: Proses bir olayın gerçekleşmesini beklemektedir (I/O, bir cihazdan geribildirim).
- Ready: Proses çalışmak için CPU'ya atanmak üzere bekliyor.
- Terminated: Proses çalışmasını sonlandırır.*



1

> 📌 ***'Soru Gelir' Alarmı:*** *Bir proses belleğe yüklendiğinde dört ana bölmeye ayrılır.*
>
> - *"Hangi bölümler dinamik olarak büyür?"* ***Cevap:*** *Stack ve Heap.*
> - *"Fonksiyon parametreleri nerede tutulur?"* ***Cevap:*** *Stack.*
> - *"Dinamik bellek ayırma nerede yapılır?"* ***Cevap:*** *Heap.*
> - *"Kodda bölgeleri işaretlenmiştir, açıkla?"* ***Cevap:*** *Heap ve stack birbirine doğru büyürler, böylece bellek daha verimli kullanılır. Ancak bu iki bölme birbirine çok yaklaşırsa, yani birbirlerinin alanını işgal etmeye başlarsa, bu durum "stack overflow" hatasına yol açar ve sistem çöker.*

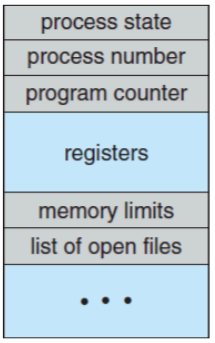
---

3. Proses Durumları (Process States)

Bir proses hayatı boyunca şu beş durumdan geçer. Bunu bir banka kuyruğu gibi hayal et:

Durum	Açıklama	Banka Kuyruğu Benzetmesi
New	Bebek proses, yeni oluşturuluyor.	Banka kapısından girmek.
Ready	"Ben hazırım, CPU boşalınca beni alın" diyenlerin sırası.	Sıra numaranı alıp beklemek.
Running	Şu an CPU'da koşan, işlem yapan proses.	Gişede işlem yaptırmak.
Waiting	Bir olayı (mes. kullanıcının tuşa basmasını) bekleyen proses.	İmza için evrak beklemek üzere kenara geçmek.
Terminated	İşi bitti, sistemden temizleniyor.	İşlemi bitirip bankadan çıkmak.

- Bir PCB'de aşağıdaki bilgiler tutulur:
  - Process ID/number : Her prosese özgün benzersiz bir proses ID/number atanır.
  - Process state: Durum bilgisi: new, ready, running, waiting, halted olabilir.
  - Program counter: Bu proses için sonraki komutun adresini gösterir.
  - CPU register'ları: Prosesle ilişki olan tüm register içerikleri tutulur. Bir interrupt olduğunda prosesin register bilgileri kaydedilir, proces tekrar running durumuna geçtiğinde bu registerlerdeki bilgiler ile en son kaldığı yerden devam eder.
  - CPU-scheduling information: Proses önceliğini içerir.
  - Memory-management information: proses için ayrılan bellek bilgilerini içerir
  - Accounting information: CPU kullanım oranları, zaman limitleri, account bilgileri ve proses numaralarını içerir.
  - I/O status information: Proseslere tahsis edilmiş I/O cihazları ile açık durumdaki dosyaları içerir.



14

---

4. PCB (Process Control Block) ve Thread

PCB, prosesin "duraklat/devam et" düğmesidir (kimlik kartı). İçinde **Program Counter** (sıradaki komut/satır), Process State (durum), açık dosyalar ve bellek bilgileri bulunur.

- **"Durdurulmamış Gibi Devam Etmek":** CPU müzikten ayrılıp Word'e geçeceği zaman Müzik prosesinin nerede kaldığını PCB'sine yazar. Tekrar ona döndüğünde PCB'yi okur ve hiç durmamış gibi devam eder.
- **Merkezi Rol:** İşletim sisteminin zamanlayıcıları bir karar vereceği zaman hemen PCB'ye bakar.
- **Not:** Linux'ta her proses, C dilindeki `struct task_struct` adı verilen bir yapı ile temsil edilir. Aktif prosesler çift yönlü bağlı liste (Doubly Linked List) içinde tutulur.

> 💡 ***Mantıksal Benzetme: "Mutfak Sipariş Fişi"*** *Aşçısın (CPU). 10 masanın siparişi (Prosesler) geliyor. Her masanın sipariş fişi (PCB) var. Bir yemeği ızgaraya attın, pişerken fişine not düşersin (Waiting). O sırada diğer masanın siparişini hazırlarsın. Köfte pişince duruma göre tabağa alırsın. Fiş (PCB) olmasa hangi masanın ne aşamada olduğunu asla hatırlayamazdın.*

Thread Kavramı

- Tek thread ile bir proses kontrol edilir ve birden fazla görev aynı anda yapılamaz (bir Word programında karakter girişi ile yazım denetleyici aynı anda yapılamaz).
- Modern işletim sistemlerinde bir proses ile birden fazla thread çalıştırılmasına izin verilir. Bu özellik multicore işlemcilerde çok faydalıdır ve birden çok görev eşzamanlı (concurrency) yapılabilir.
- Birden çok thread ile çalışan sistemlerde, PCB ile her bir thread'e ait bilgiler saklanır.

---

5. Multiprogramming ve Time-Sharing

Terim	Temel Amaç	Özellik
Multiprogramming (Çoklu Prog.)	CPU kullanımını maksimize etmek.	Proses I/O beklerken, CPU hemen başka prosese geçer.

Time-Sharing (Zaman Paylaşımı)	Kullanıcı etkileşimi (interaction) sağlamak.	Geçişler o kadar hızlıdır ki kesintisiz çalışma hissi verilir.

Process Scheduler (Zamanlayıcı)

CPU'nun başına kimin geçeceğine karar veren "trafik polisi"dir. Bu mekanizma sayesinde sistem verimli çalışır.

> 📌 ***'Soru Gelir' Alarmı:***
>
> - ***Soru:*** *Dört çekirdekli (quad-core) işlemcide aynı anda en fazla 1 proses mi yürütülebilir?* ***Cevap:*** *Yanlış. Her CPU çekirdeğinde 1 proses çalışabilir, bu yüzden multicore bir sistemde aynı anda birden çok proses çalışabilir.*
> - ***Soru:*** *Zaman paylaşımlı sistemlerde çok sık yer değiştirilmesinin temel sebebi nedir?* ***Cevap:*** *Kullanıcının her programla etkileşime girebilmesini sağlamaktır.*
> - ***Soru:*** *Bir proses CPU'da çalışırken "klavyeden bir veri gelmesini" beklediğinde, CPU boşa gitmesin diye başka prosese geçilmesine ne ad verilir?* ***Cevap:*** *Multiprogramming.*

---

6. Process Scheduling (Zamanlama Kuyrukları)

Bir proses farklı kuyruklara alınabilir, bu seçim **scheduler** (zamanlayıcı) tarafından gerçekleştirilir.

Kuyruk Adı	İşlevi	Benzetme
Job Queue (İş Kuyruğu)	Sistemdeki tüm proseslerin listesidir.	Hastaneye kayıt yaptıran herkes.
Ready Queue (Hazır Kuy.)	Sadece CPU'yu bekleyen, hafızaya yüklenmiş prosesler.	Her şeyiyle hazır, muayene odasına girmeyi bekleyen hasta.
Device Queue (Bekleme/Cihaz)	I/O (klavye/disk vb.) bekleyen prosesler.	Röntgen cevabını bekleyen hasta.

- **Long-term scheduler (job scheduler):** Diskteki işleri seçerek hafızaya yükler. Dakikalık aralıklarla çalışır.İş kuyrugunda hangı processlerin hazır kuyruguna gecerıne karar verir. CPU'ya gelen proseslerin sayısını kontrol eder, böylece multiprogramming derecesini belirler.
- **Short-term scheduler (CPU scheduler):** Hazır olanları seçerek CPU'yu onlara tahsis eder. Çok sık (<100ms) çalışır.
- **Medium-term scheduler:** Bellek aşırı yüklendiğinde bazı prosesleri geçici olarak diske atar (swapping) ve CPU için rekabeti azaltır.

CPU-bound vs I/O-bound Prosesler

Proses Tipi	Özellik	Restoran Benzetmesi
I/O-bound	Bekleme süresi çok, hesaplama az. Zamanını I/O bekleyerek geçirir.	Yemeğini yemiş, kahvesini yudumlayan (garson bekleyen) müşteri.
CPU-bound	Hesaplama çok, bekleme az.	Çok aç gelmiş, sürekli yemek yiyen ve mutfağı meşgul eden müşteri.

Not: Sistemin dengeli çalışması için zamanlayıcı bu ikisini karışık (mix) seçmelidir. Sadece CPU-bound proses varsa I/O birimleri boş kalır, israf olur.

Swapping ve Medium-Term Scheduler

Bellek aşırı yüklendiğinde yer açmak (sistemi rahatlatmak) için bazı prosesleri geçici olarak diske/HDD'yekoridora çıkarmak işlemine **Swapping** denir. Bu işlemi **Medium-term Scheduler** yapar. Bu işlem sonucunda CPU için rekabet edemez hale gelir, ve **multiprogramming derecesi düşürülür.**

---

7. Context Switch

Bir prosesin CPU'da yürütülürken başka bir prosesin CPU'ya geçmesine karar verildiğinde çıkacak prosesin (context) bilgilerinin kaydedilmesi ve CPU'ya geçecek prosesin bilgilerinin yüklenmesine denir.

- Context bilgisi **PCB** içerisinde saklanır.
- Context switch süresi, bu geçişte bir iş üretilmediği için **Overhead (ek yük)** olarak adlandırılır. Donanımın kalitesi süreyi kısaltır.
- Bazı donanımlarda her CPU için birden çok register seti sağlanır ve pointer'in değiştirilmesiyle daha pratik yapılır.

PCB ve Context Switch Arasındaki İlişki

- **PCB (Process Control Block):** Her prosesin kendine ait bir PCB'si vardır. Bu blok, prosesin durumunu, program sayacını, CPU register'larını, açık dosyaları ve diğer önemli bilgileri içerir.
- **Context Switch:** Bir proses CPU'dan çıkarılırken, o prosesin PCB'sindeki bilgileri kaydedilir. Yeni proses CPU'ya geldiğinde, onun PCB'sindeki bilgiler yüklenir. Bu sayede her proses kendi durumunu korur ve kesintisiz çalışıyormuş gibi devam eder.

Soru: Context switch sırasında hangi bilgiler kaydedilir ve yüklenir? **Cevap:** Program sayacı (PC), CPU register'ları, proses durumu, açık dosyalar ve diğer önemli bilgiler PCB'de saklanır ve context switch sırasında bu bilgiler kaydedilir ve yüklenir.

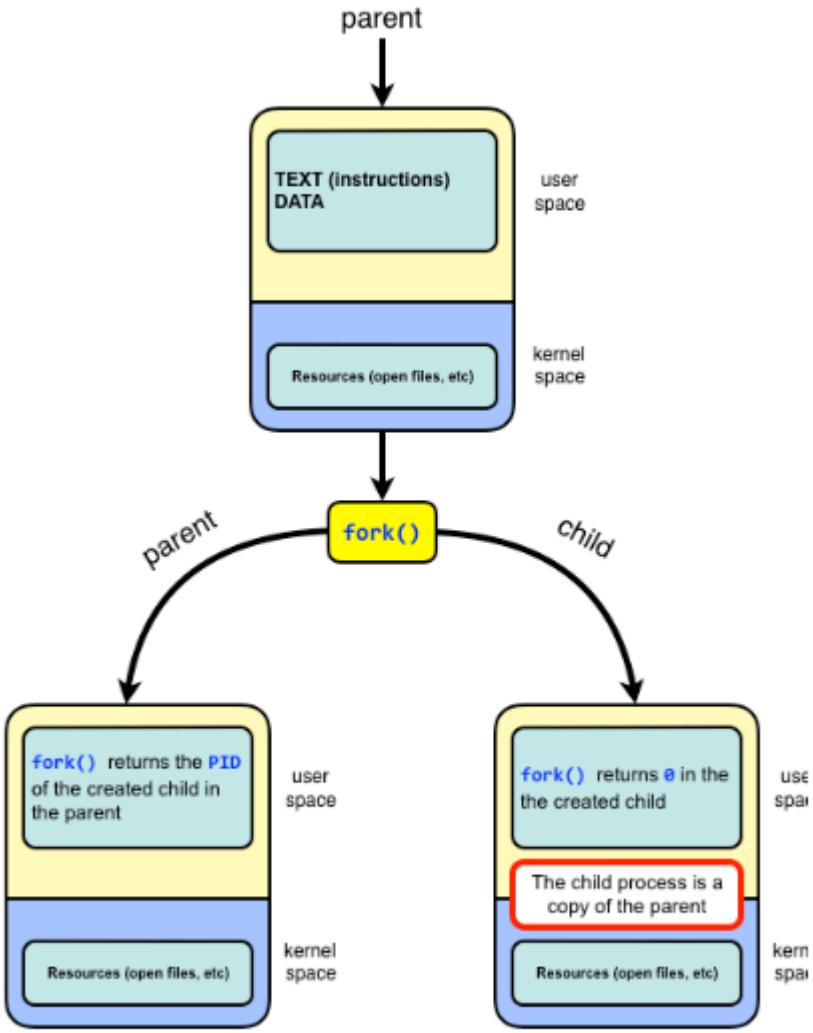
Soru: Context switch süresi neden overhead olarak adlandırılır? **Cevap:** Çünkü context switch sırasında CPU'nun gerçek bir iş yapmadığı, sadece prosesler arasında geçiş yaptığı için ek bir yük (overhead) oluşturur. Bu süre, sistem performansını etkileyebilir ve mümkün olduğunca kısa tutulmalıdır.

---

8. Proseslerin Doğumu (Process Creation)

- **Anne-Çocuk İlişkisi:** Bir proses (Parent), başka yeni prosesler (Child) oluşturabilir (Process Tree oluşur).
- **PID (Kimlik):** Her prosese benzersiz PID verilir. Kaynaklar OS tarafından tahsis edilebilir veya babasından aktarılabilir.

> 🎉 ***"fork()" Fonksiyonunun "Sihirli" Dönüş Değeri*** `fork()` *çağrıldığında sistemde iki tane olay oluşur ama dönüş değerleri farklıdır:*
>
> - ***Parent (Baba):*** *Yeni oluşturulan çocuğun PID numarasını döndürür. (0'dan büyüktür).*
> - ***Child (Çocuk):*** *0 döndürür.*
> - ***Hata Durumu:*** *Eğer -1 dönerse, çocuk oluşturulamamış demektir.*

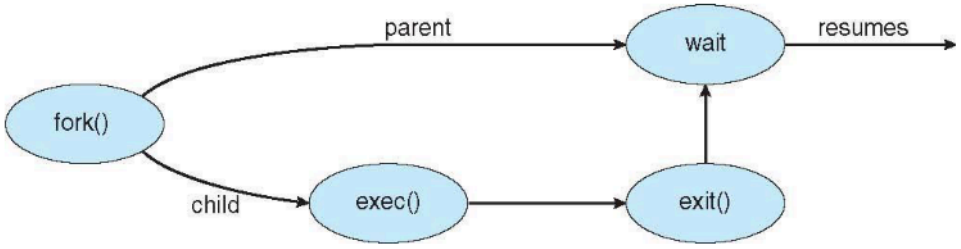

parent
TEXT (instructions)
DATA
user
space
Resources (open files, etc)
kernel
space
fork()
parent
child
fork() returns the PID
of the created child in
the parent
user
space
Resources (open files, etc)
kernel
space
fork() returns 0 in the
the created child
The child process is a
copy of the parent
Resources (open files, etc)

```c
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

int main()
{
pid_t pid;

    /* fork a child process */
    pid = fork();

    if (pid < 0) { /* error occurred */
       fprintf(stderr, "Fork Failed");
       return 1;
    }
    else if (pid == 0) { /* child process */
       execlp("/bin/ls","ls",NULL);
    }
    else { /* parent process */
       /* parent will wait for the child to complete */
       wait(NULL);
       printf("Child Complete");
    }

    return 0;
}
```

35

Neden if-else yapısı var?

- `pid == 0` (Çocuk tarafı): Çocuğa `execlp()` gibi bir komutla yeni görev verilir.
- `else` (Baba tarafı): Baba `wait(NULL)` komutunu kullanır. Çocuk bitirmeden baba devam ederse çocuk "yetim" kalmasın diye bekler. `wait` komutu, prosesin bitmesini bekler ve kaynakları temizler.



---

9. Proseslerin Ölümü (Termination)

1. **Gönüllü Ölüm (`exit()`):** İşini bitirip silinmeyi ister. Durum bilgisini `wait()` ile babasına iletir.
2. **Zoraki Ölüm (`abort()`):** Baba çocuğu zorla sonlandırabilir (örn. kaynak tüketiyorsa).
3. **Kademeli Sonlandırma (Cascading Termination):** Baba öldüğünde sistemin tüm çocuklarının da otomatik öldürülmesidir.

Zombie ve Orphan (Zombi ve Yetim) Proses

Kavram	Durum	Ne Olur?	Restoran Benzetmesi
Zombie (Zombi)	Çocuk öldü, babası henüz `wait()` demedi.	Tabloda giriş kaplar. Baba `wait()` dediğinde tamamen yok olur.	Garson gelmeden temizlenmeyen, kirli masanın dolu gözükmesi.
Orphan (Yetim)	Çocuk yaşıyor, ama babası hesabı ödemeden (wait() çağırmadan) öldü.	`init` (root) prosesi periyodik `wait()` çağırıp onu evlat edinir ve temizler.	Babanın hesabı ödemeden kaçıp, restoran sahibinin hesabı devralması.

> 🧠 ***Şifreleme: "BA-YE-ZO-BE"*** *= **BA**ba öldüyse çocuk **YE**tim (Orphan) kalır, **ZO**mbi ise çocuk bitti ama **BE**klemededir. Soru: "Bütün prosesler kısa süreliğine zombi olur mu?" Evet.*

---

10. Chrome'un Üç Silahşörü (Proses Türleri)

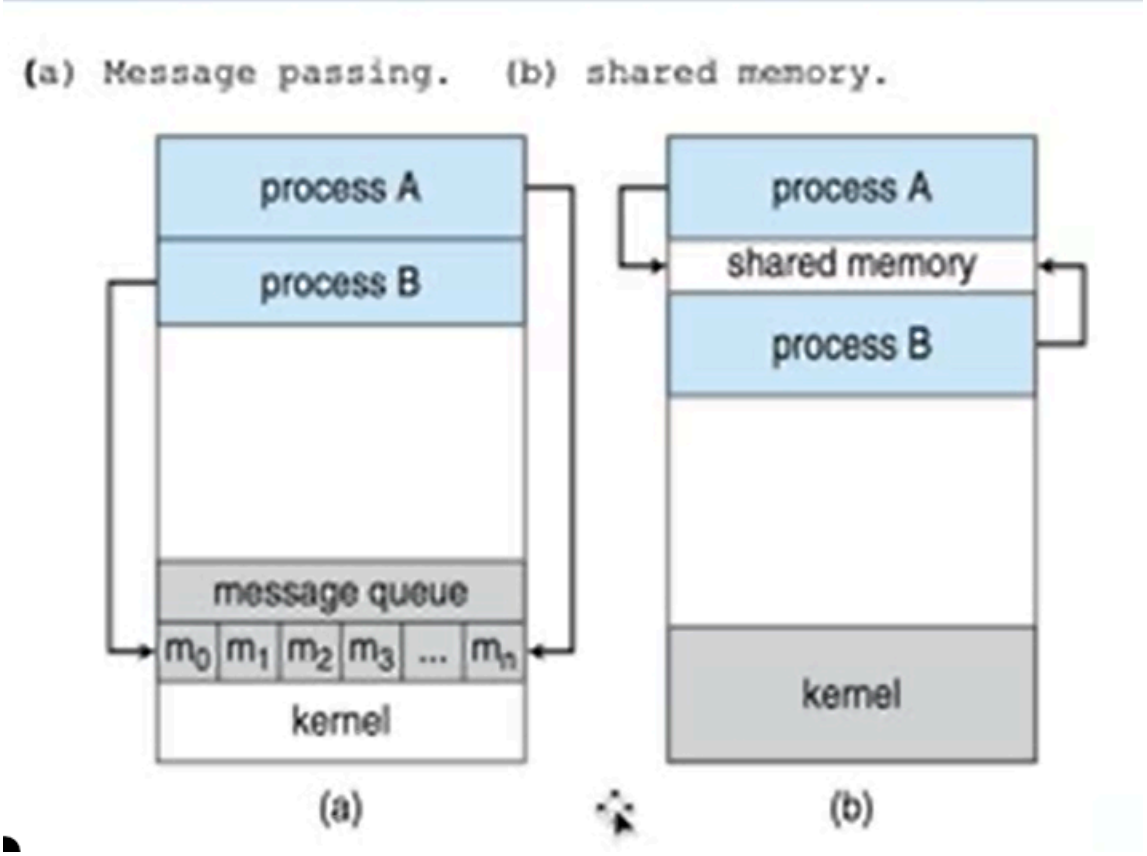
1. **Browser Process (Ana Yönetici):** Kullanıcı arayüzü, diski ve ağı yönetir.
2. **Renderer Process (Görselleştirici):** Sayfa HTML ve JavaScript içini işler. Açtığın her sekme için ayrı oluşturulur.
3. **Plug-in Process (Eklenti):** Flash veya eklentiler için ayrı proses.

- **Avantaj: Hata İzolasyonu** (sekme çökerse diğerleri sapasağlam kalır) ve **Güvenlik/Sandbox** (ağ/disk kısıtlı kum havuzunda virüs engellenir).

---

11. Prosesler Arası İletişim (IPC)

Model	Mantık	Hız & Özellik
Shared Memory (Paylaşılan Bellek)	Hafızada ortak bölge (not defteri) belirlenir.	**Çok hızlıdır.** "Aynı anda yazma" sorunu (conflict) vardır, senkronizasyon gerekir.
Message Passing (Mesajlaşma)	Prosesler birbirine mesaj (send/receive) atar.	**Düşük hız**, çünkü proses kernel (çekirdek) araya girer. Dağıtık ağ sistemlerinde çok iyi ve güvenlidir.



Producer-Consumer (Üretici-Tüketici) Problemi

Paylaşılan bellek mantığının en somut örneğidir.

- **Producer (Üretici):** Veri üreten proses (Örn: Fırıncı).
- **Consumer (Tüketici):** Veriyi kullanan proses (Örn: Müşteri).
- **Buffer (Tampon Bölge):** Ortada beklenen depo (Örn: Ekmek Dolabı).
- **Senkronizasyon şarttır:** Üretici depo doluyken taşmamalı, tüketici depo boşken olmayan malı almaya çalışmamalı.
- Producer item next_produced; while (true) { /* produce an item in next produced */ while (((in + 1) % BUFFER_SIZE) == out) ; /* do nothing */ buffer[in] = next_produced; in = (in + 1) % BUFFER_SIZE; } 55

- Consumer item next_consumed; while (true) { while (in == out) ; /* do nothing */ next_consumed = buffer[out]; out = (out + 1) % BUFFER_SIZE; /* consume the item in next consumed */ } 56

Buffer Türleri:

1. **Unbounded Buffer (Sınırsız Depo):** Boyutu pratikte sınırsızdır. Üretici hiç beklemez, her zaman yeni mal üretir. Tüketici sadece boşsa bekler.
2. **Bounded Buffer (Sınırlı Depo):** Sabit kapasite vardır. Depo doluysa üretici bekler. Depo boşsa tüketici bekler.